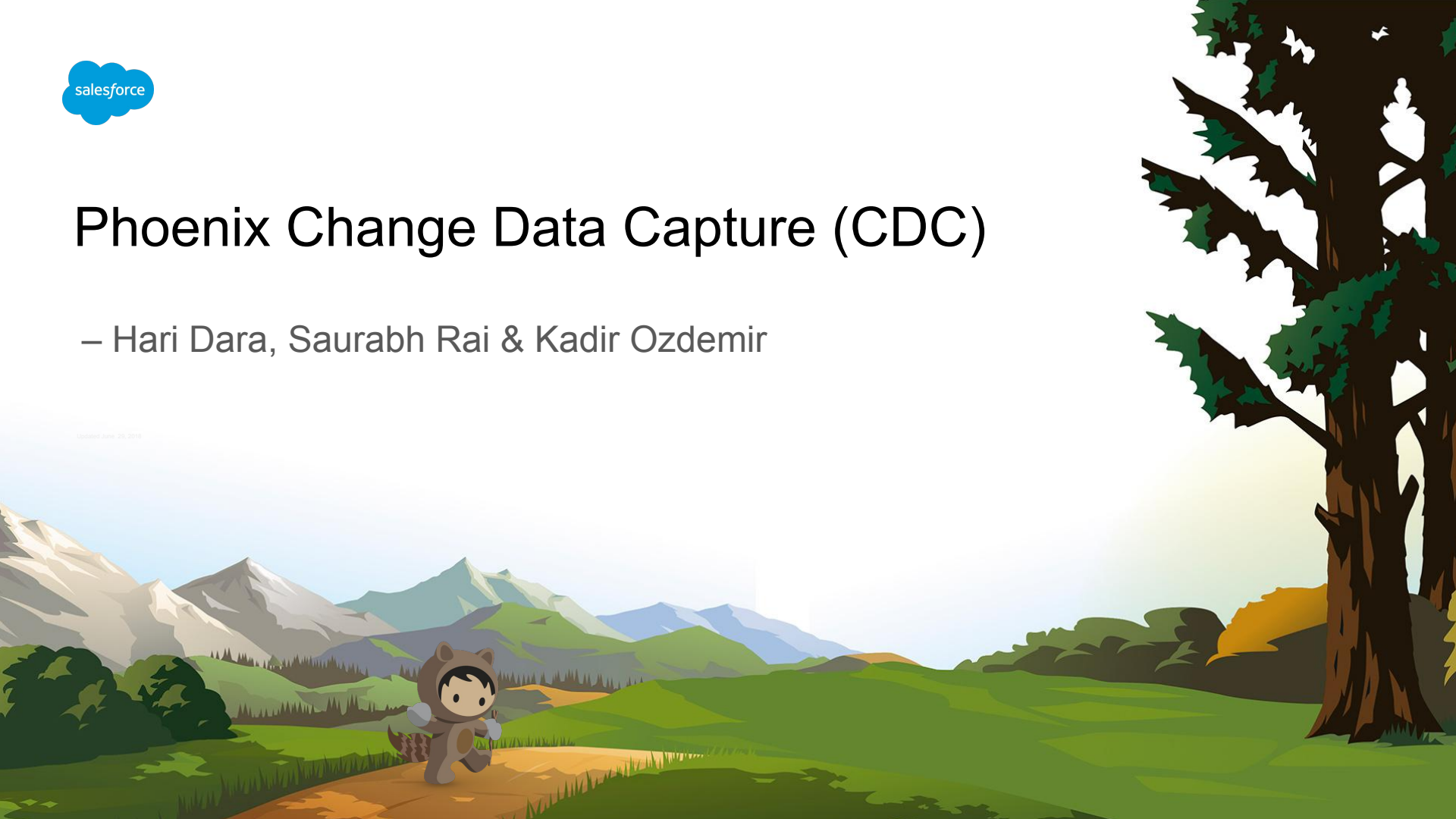
Phoenix Change Data Capture (CDC)

– Hari Dara, Saurabh Rai & Kadir Ozdemir




THANK YOU

Agenda

- Introduction
- Features and Architecture
- SQL syntax
- Querying CDC
- Demo
- Limitations
- Status

What is it?

- Capture changes to tables or updatable views
- Real-Time Change Retrieval
  - Capture and retrieve changes as they happen or with minimal delay
- Comprehensive Change Tracking
  - Track UPSERTs (inserts and updates) and DELETEs
  - Track the row state before and after the UPSERT.
  - Track the row state before the DELETE.

What is it not?

- CDC in Core
  - A Phoenix customer to support CDC for Oracle tables
  - CHANGE_DATA_CAPTURE.* tables
  - See: Log-Based Change Detection Architecture
- CDC based on event stream
  - An older design that is based on Kafka topic pub/sub
  - See: CDC resources

Key Features

- Ordered Change Delivery
  - Chronological sequence is maintained
- Familiar interface
  - Changes retrieved using SQL syntax
  - Support for basic query filters
- Control the information retrieved
  - Use change types to control the inclusion of pre/post/change images
- Minimal storage overhead
- Minimal setup
  - Most other databases require complex setup such as WAL replay or replication

Metadata

- New PTableType called `CDC`
  - `TABLE_TYPE = 'c'` in SYSCAT
- Virtual table with predefined columns
  - No physical HBase table
  - `PHOENIX_ROW_TIMESTMAP()` as the first column
  - PK columns from data table are mirrored
  - Additional `"CDC JSON"` column of `VARCHAR` type
- New SYSCAT column: `CDC_INCLUDE`
  - Stores the default change types to include, specified at the time of creation
- CDC index is set as physical table name
  - Link from CDC table to CDC index with LINK_TYPE of 1 (LinkType.PHYSICAL_TABLE)

Mechanics

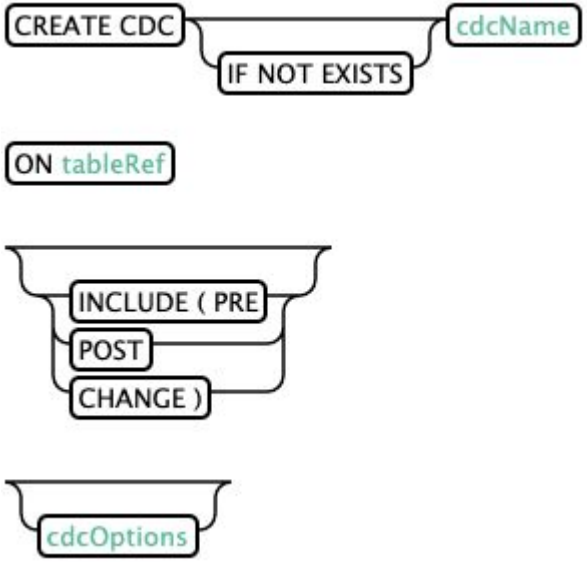
- CDC object has no physical HBase table of its own
- When a CDC is created, a CDC index is also implicitly created
- The CDC index acts as the physical table of CDC
- CDC index is a regular uncovered index that indexes data table on

  `PHOENIX_ROW_TIMESTAMP()`

  - Regular except for some additional delete markers.
- All UPSERTs and DELETEs on data table result in a new row in the CDC index.
- Index rows get deleted per TTL and MAX_LOOKBACK rules on the data table.
- SELECT queries against the CDC object result in a query against the index

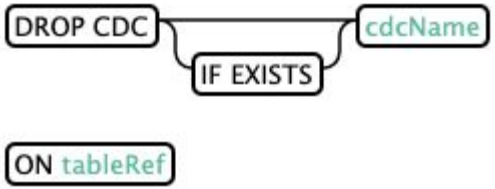
DDL to create

- `cdcName` is a unique identifier, like a table name.
- `tableRef` is like in `CREATE INDEX`.
- `INCLUDE` spec is optional
- `cdcOptions` is like `tableOptions but only valid for:`
  - `SALT_BUCKETS`
  - `COLUMN_ENCODED_BYTES`
  - `IMMUTABLE_STORAGE_SCHEME`
- Creates the CDC object and the underlying index.
  - CDC will be ready for queries after index is fully built.



DDL to drop

- Syntax is very similar to `DROP INDEX`.
- Drops the CDC object as well as the underlying index



Querying


```
SELECT [/*+ CDC_INCLUDE(...) */] <projections> FROM <CDC-name> [WHERE
<clauses> ORDER BY <clauses>]
```

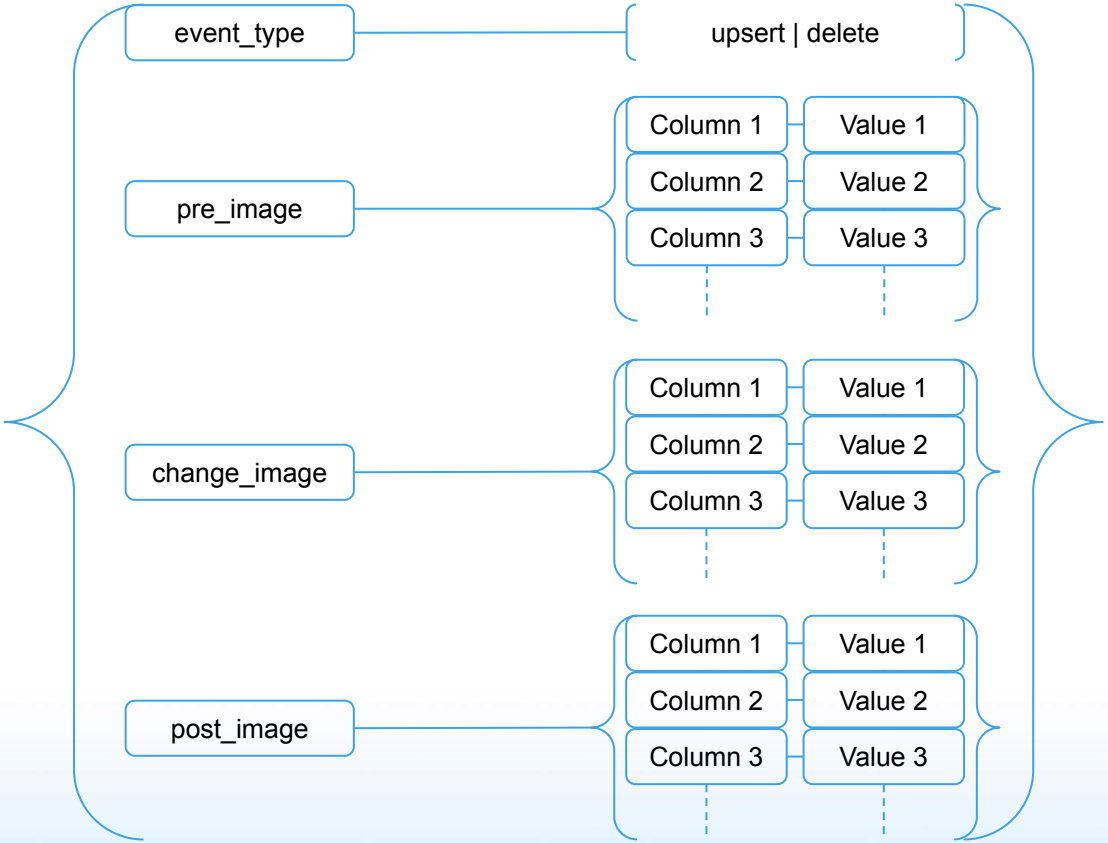
- `SELECT * FROM <CDC-name>` will return all changes in the chronological order
- Instead of `*`, we can specify the columns defined on the CDC.
- Use `WHERE` clause to filter changes by change timestamp `PHOENIX_ROW_TIMESTAMP()`
- Use the hint to specify the comma separated list of change types to include in `"CDC JSON"`

"CDC JSON" format

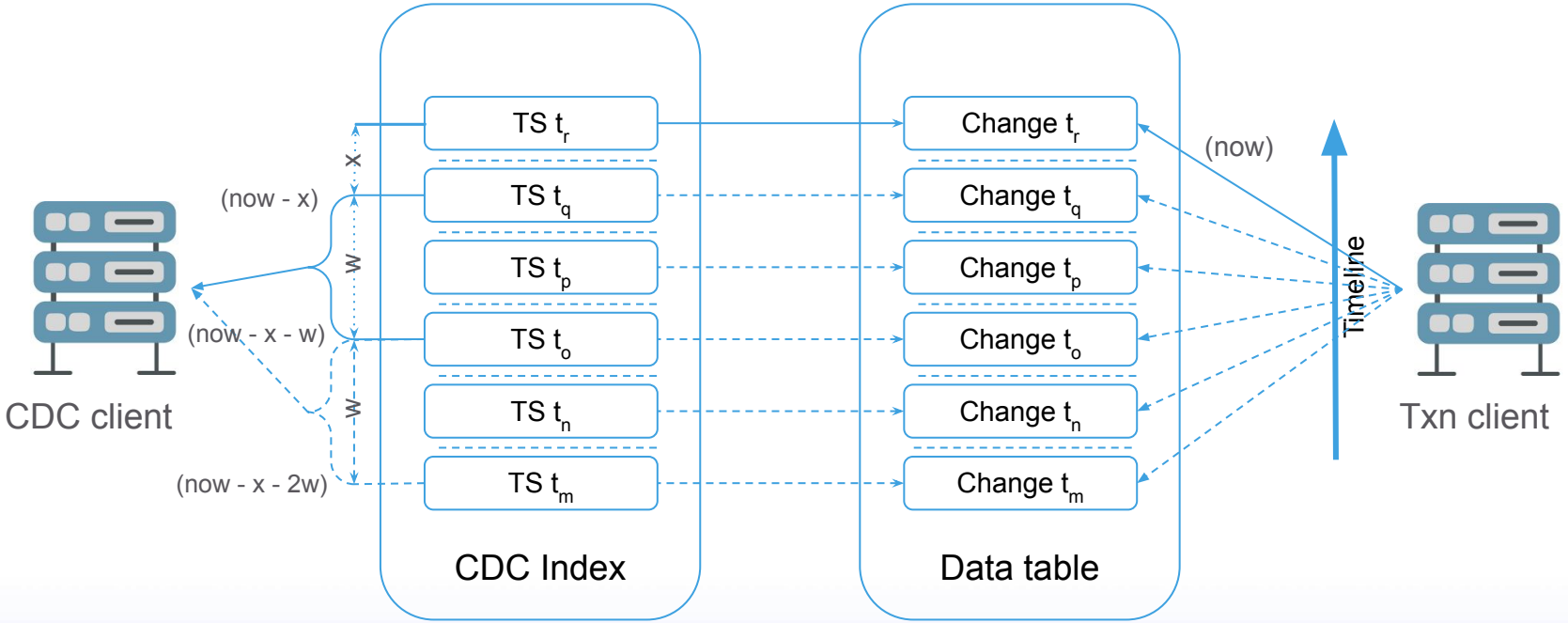
The `"CDC JSON"` column contains a JSON map containing the following top level fields:

- `event_type`: The type of event containing the string value `"delete"` when the row is deleted, otherwise `"upsert"`.
- `pre_image` (optional): The state of the row before the change as a dictionary of column names and their values. This image will be included only if the `INCLUDE` specification contains `PRE` value.
- `change_image` (optional): The exact columns that changed, as a dictionary of column names and their values. This image will be included only if the `INCLUDE` specification contains `CHANGE` value.
- `post_image` (optional): The state of the row after the change as a dictionary of column names and their values. This image will be included only if the `INCLUDE` specification contains `POST` value.

"CDC JSON" format (cont.)



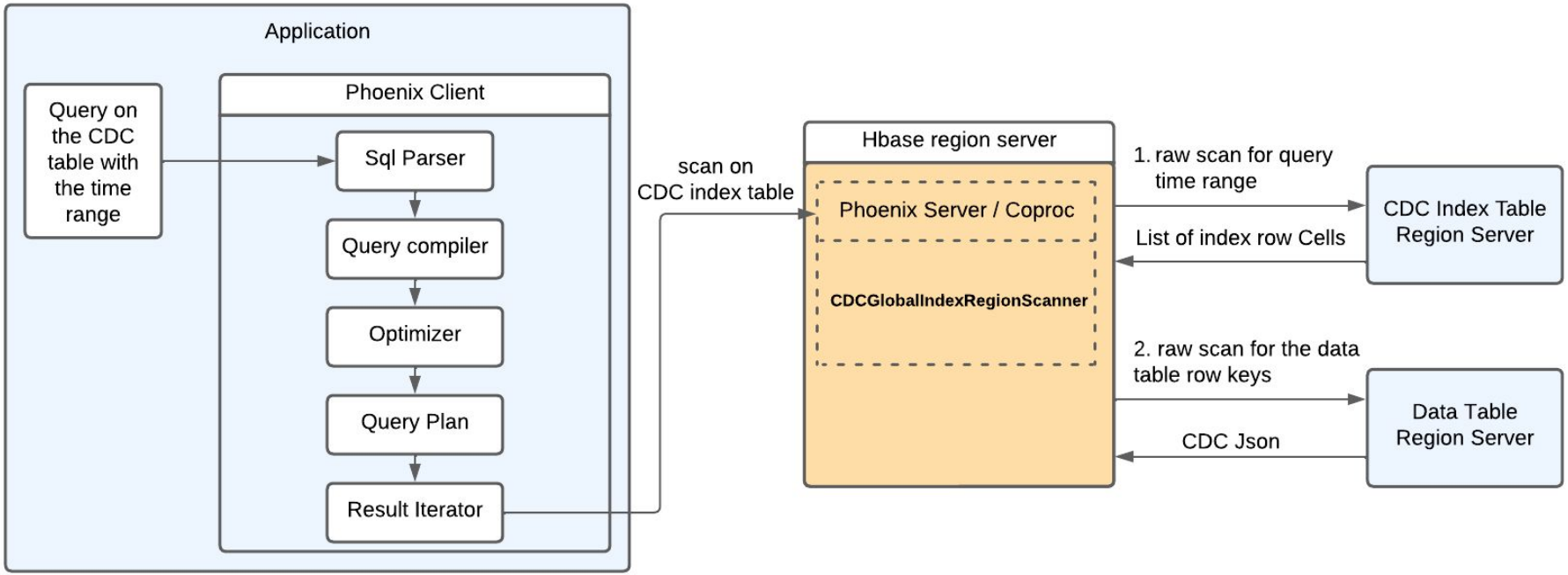
Query Pattern for Streaming



Query Pattern for Streaming (cont.)

- Create a continuous sliding time window using `WHERE` clause on `PHOENIX_ROW_TIMESTAMP()`
- Keep track of the last read timestamp (ts) as a pointer.
- Keep the window (w) small enough to have a good balance between latency and throughput.
- Ensure no overlap between windows to avoid seeing a change multiple times.
- Ensure that the time window doesn't extend into future to avoid missing changes.
- Account for time skew by ensuring that upper bound of the time window is trailing the current time, say x, where x can be as small as 1000 milliseconds.

Phoenix CDC Architecture (Read Path)



Phoenix CDC Algorithm

Phoenix Client:

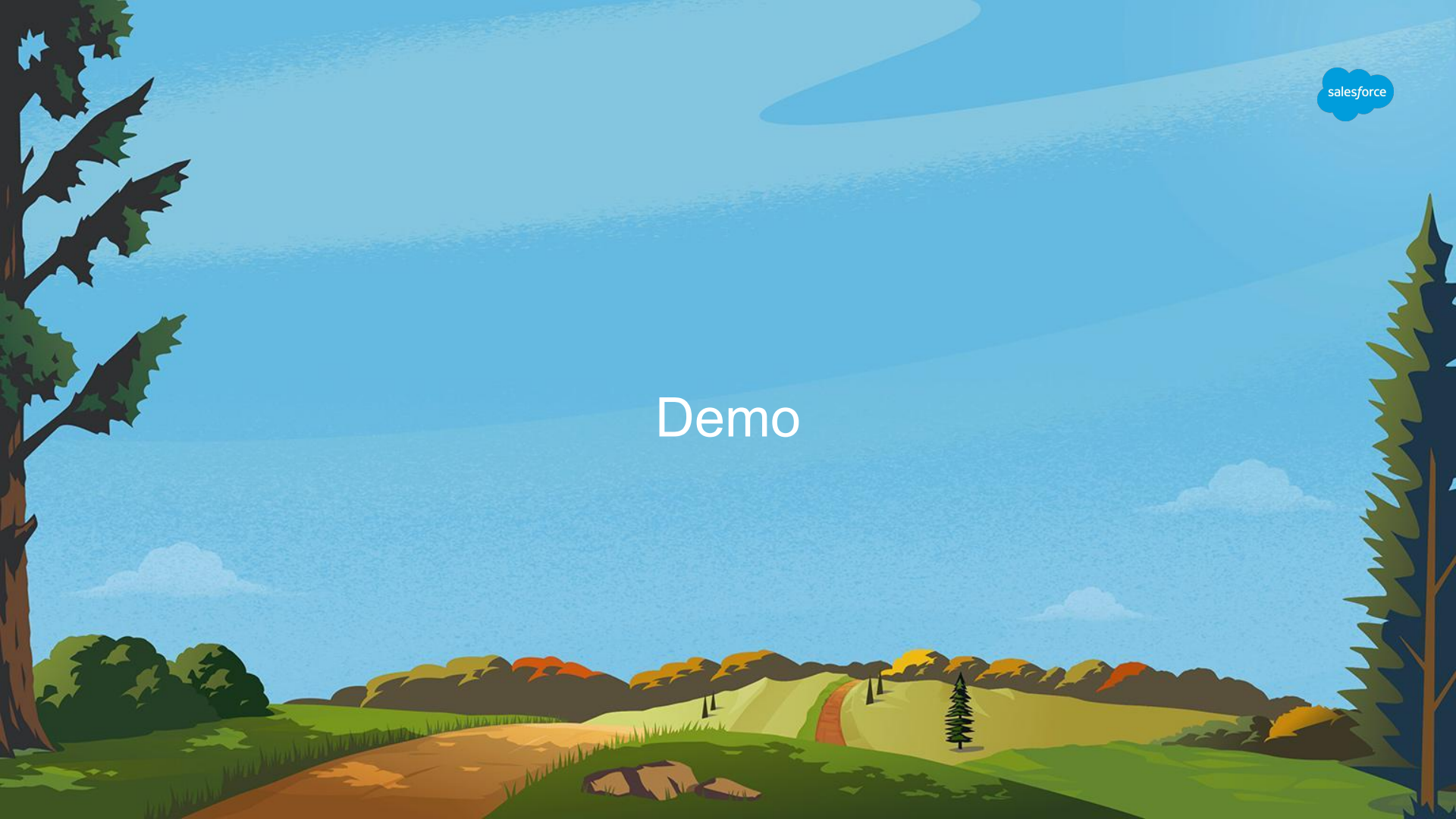
1. Runs a query on the CDC (Change Data Capture) table within a specific time range.

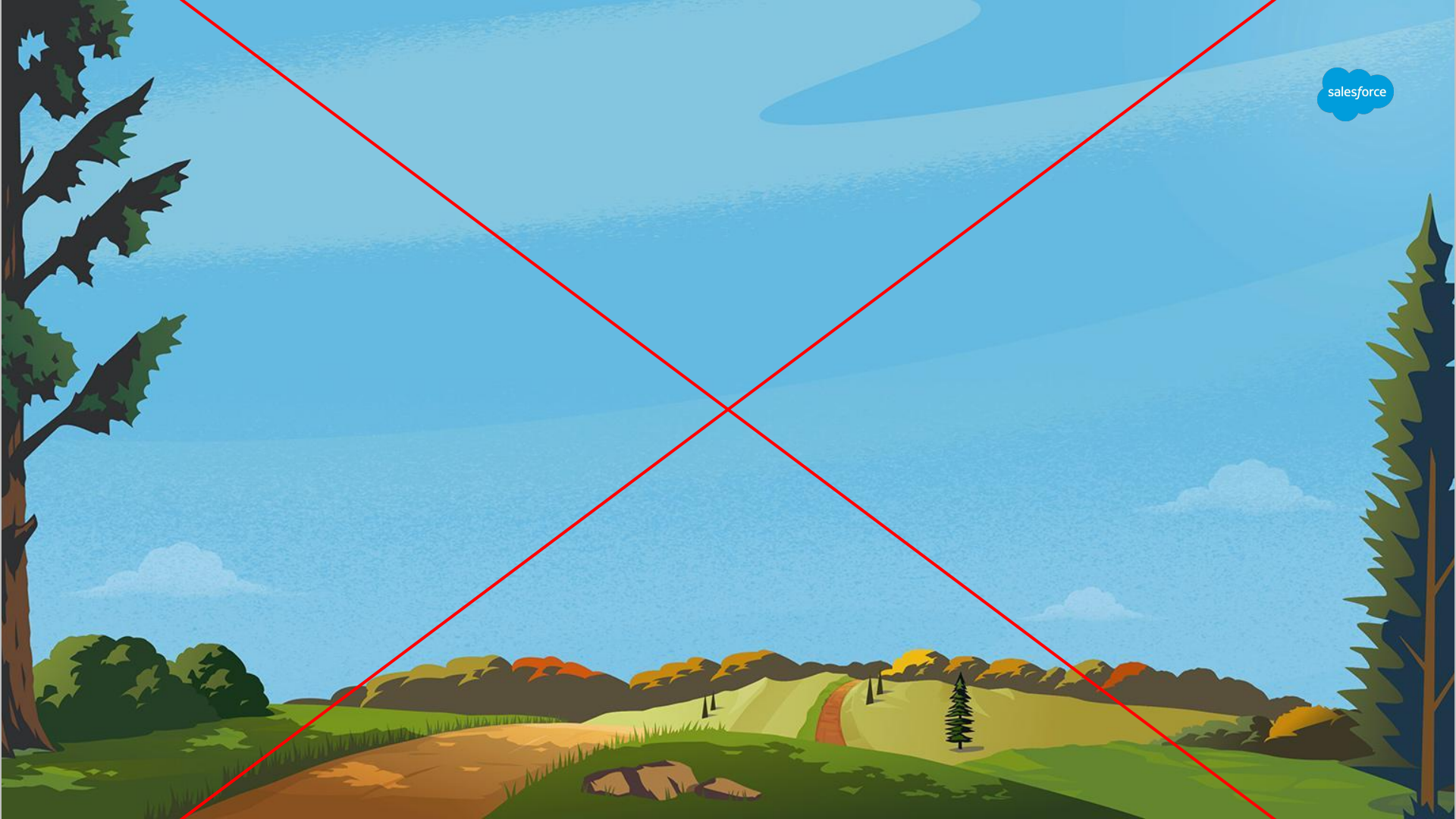
Phoenix Server / HBase Region Server:

2. Scans the CDC index table for entries matching the time range.
3. Builds a map (`indexToDataRowKeyMap`) linking each index row key to its corresponding data row key.
4. Processes each index row one by one:
    a. Scans the data table using the data row key and the time range.
    b. Goes through each cell in the data row.
    c. Uses timestamps and cell types from both the index and data row to build the event.
    d. Creates a CDC event object for each cell, including:
        i. Event type (Delete / Upsert)
        ii. Change, Pre-change and post-change data
5. Sends each CDC event object to the Phoenix client as it is created.

Phoenix Client (continued):

6. Receives the CDC event object and continues calling `next` on the `ResultScanner` until all relevant rows are processed and collected.


salesforce
Demo


salesforce

CDC Write Perf results (latency comparison)

Table Schema –

```
PK_BUCKET VARCHAR NOT NULL,
RUUID VARCHAR NOT NULL,
R_VALUE VARBINARY,
CONSTRAINT PK PRIMARY
KEY(PK_BUCKET, RUUID)
```

Row Size = 512 kb
Total write Agent = 8
No. of Rows / agent (warm-up) = 16 (threads) * 100000 (rows)
No. of Rows / agent (actual) = 16 (threads) * 500000 (rows)

CDC Enabled	Salt (CDC table)	Avg (ms)	P90 (ms)	P95 (ms)	P99 (ms)	Max (ms)
FALSE		5.7	6.5	7.8	19.5	37.3
TRUE	0	8.8	9.5	17	34	49
TRUE	24	7.4	7.9	13	30	56
TRUE	48	6.9	7.5	8.2	20	55

Analysis

1. Enabling CDC generally increases latency across all measured percentiles (P90, P95, P99) and maximum latency.
2. Salt values of 0, 24, and 48 were used, with higher salt values generally showing a trend of decreasing average latency and percentile latencies compared to no salt

CDC Read/Write throughput comparison

Objective

Evaluate **write** and **read** operations on a **CDC-enabled datatable** using the **Phoenix Scenario Executor**.

Test 1

Time (min)	Type	No. of Agent & Threads	total rows (write/read)	duration (s)	Spec
T	Write	3 & 16	1.3 mil	~300 - 320	Initial writing phase
T + 5	Write	3 & 16	1.7 mil	~300 - 320	Sustained throughput.
T + 10					
	Write	3 & 16	1.76 mil	~300 - 330	Parallel writes and reads start.
	Read	1 & 1	1.6 mil	~62 s	scan range = 5 min

Test Setup

- **Write Operations**:
  - Agents: 3
  - Threads/Agent: 16
  - Rows/Thread: 10,000 (running in loops for 5 minutes)
- **Read Operations**:
  - Agents: 1
  - Scan Interval: 5 minutes.
- **Data Details**:
  - **Column Size**: 512 bytes
  - **Salting**: Enabled on the CDC Table, **Salt Value: 24**

Total write threads = 48
Total Read threads = 1

Read Efficiency: Processed 1.6 million rows in **62 seconds**.
Parallel Operations: Sustained high write throughput alongside efficient read operations.

CDC Read/Write throughput comparison

Test 2

Time (min)	Type	No. of Agent & Threads	total rows (write/read)	duration (s)	Spec
T	Write	7 / 16	~7.6 mil	~600-650	Initial writing phase.
T + 10	Write	7 / 16	~7.6 mil	~600 - 650	Sustained throughput.
T + 20					
	Write	7 / 16	6 mil	~600-650	Parallel writes and reads start.
	Read	5 / 1	1.72 mil + 1.72 mil + 1.72 mil + 1.72 mil + 1.72 = 8.5 mil	~162 && ~161 && ~170 && ~162 && ~165	scan range = 2 min per agent = 5 * 2 = 10 min

Test Setup

- **Write Operations**:
  - Agents: 7
  - Threads/Agent: 16
  - Rows/Thread: 10,000 (running in loops for 10 minutes)
- **Read Operations**:
  - Agents: 5
  - Scan Interval: 2 minutes.
- **Data Details**:
  - **Column Size**: 512 bytes
  - **Salting**: Enabled on the CDC Table, **Salt Value: 24**

Total write threads = 112
Total Read threads = 5

Read Efficiency: Processed 8.5M rows across 5 agents with balanced scan times (~162–170 seconds).

CDC Read/Write throughput comparison

Test 3

Time (min)	Type	No. of Agent & Threads	total rows (write/read)	duration (s)	Spec
T	Write	10 / 48	~24.96 mil	~600-720	
T + 10	Write	10 / 48	~24.96 mil	~600 - 700	
	Read	5 / 1	4736862 + 4716370 + 4422091 + 4482060 + 1704165 = ~20.06 mil	~800 - 850	scan range = 2 min per agent = 5 * 2 = 10 min
T + 20	Write	10 / 48	~26.4 mil	~600-720	
	Read	5 / 1	4698740 + 4660669 + 4507267 + 3115783 + 0 = ~16.9mil	~670-750	scan range = 2 min per agent = 5 * 2 = 10 min
T + 30	Write	10 / 48	~24.48 mil	~600-720	
	Read	5 / 1	4813858 + 4796701 + 4803588 + 3748379 + 217284 = ~18.37 mil	~900-1050	scan range = 2 min per agent = 5 * 2 = 10 min

- **Write Operations**:
  - Agents: 10
  - Threads/Agent: 48
  - Rows/Thread: 10,000 (running in loops for 10 minutes)
- **Read Operations**:
  - Agents: 5
  - Scan Interval: 2 minutes.
- **Data Details**:
  - **Column Size**: 512 bytes
  - **Salting**: Enabled on the CDC Table, **Salt Value: 24**

Read Summary -

The read operation showed efficient scanning. The fixed scan interval of 10 minutes and the use of a smaller number of agent for reading demonstrate the system's ability to handle substantial read loads in a relatively short duration, even as the write operations continued in parallel.

CDC Write Heavy Throughput Summary

Scenario	Sub scenario	Total agent * Total threads	Total rows written	Total time	row size	CDC salt	throughput (rows / min)	throughput (rows / min / rs)	throughput (rows/ min / agent)
write with num of Rows as constant		12 * 64 = 768	384 million	4 hours	1 MB	48	1.6 million / min	66000 / min / rs	133000 / min / agent
write with four 10 min intervals (1)		12 * 64 = 768			512 kb	24			
	1st 10 min		~26.8 mil	10 - 12 min			2.2 - 2.68 million / min	92000 - 108000 / min / rs	183000 - 224000 / min / agent
	2nd 10 min		~28.1 mil	10 - 12 min			2.3 - 2.8 million / min	100000 - 116000 / min / rs	188000 - 233000 / min / rs
	3rd 10 min		~30 mil	10 - 12 min			2.5 - 3 million / min	104000 - 124000 / min / rs	208000 - 250000 / min / agent
	4th 10 min		~29.5 mil	10 - 12 min			2.5 - 3 million / min	104000 - 124000 / min / rs	208000 - 250000 / min / agent
write with four 10 min intervals (2)		15 * 64 = 960			512 kb	24			
	1st 10 min		~30 mil	10 - 12 min			2.5 - 3 million / min	104000 - 124000 / min / rs	166,000 - 200000 / min / agent
	2nd 10 min		~35 mil	10 - 12 min			2.9 - 3.5 million / min	120000 - 145000 / min / rs	193000 - 233000 / min / agent
	3rd 10 min		~32 mil	10 - 12 min			2.66 - 3.2 million / min	110000 - 133000 / min / rs	173000 - 213000 / min / agent
	4th 10 min		~33 mil	10 - 12 min			2.75 - 3.3 million / min	114000 - 137000 / min / rs	183 - 220000 / min / agent

CDC Write Heavy Throughput Summary

Scenario	Sub scenario	Total agent * Total threads	Total rows written	Total time	row size	CDC salt	throughput (rows / min)	throughput (rows / min / rs)	throughput (rows/ min / agent)
write with four 10 min intervals (3)		18 * 64 = 1152			512 kb	24			
	1st 10 min		~30 mil	10 - 12 min			2.5 - 3 million / min	104000 - 124000 / min / rs	138,000 - 166000 / min / agent
	2nd 10 min		~31.3 mil	10 - 12 min			2.6 - 3.1 million / min	108000 - 130000 / min / rs	144000 - 172000 / min / agent
	3rd 10 min		~35.8 mil	10 - 12 min			2.9 - 3.58 million / min	120000 - 148000 / min / rs	161000 - 198000 / min / agent
	4th 10 min		~33 mil	10 - 12 min			2.75 - 3.3 million / min	114000 - 137000 / min / rs	152000 - 183000 / min / agent

- Throughput peaks at around 15*64 = 960 threads.
- Throughput per resource and per thread shows a more gradual improvement across intervals, especially in setups with higher thread counts.

Limitations

- Schema changes
  - No special support for schema changes
- There is no way to subscribe
  - Client needs to poll by maintaining timestamp as pointer.
- Support for complex data types
  - Complex data types are treated like binary

Status & Future Roadmap

- Merged to the master branch in open source
- Index rebuild is being removed
  - Historical changes will not be captured
- Extensive tweaking for DynamoDB use case
  - Index partitioning scheme to eliminate salting
  - SQL changes

Q & A

Resources

- Design doc
- Perf results